

Phase 4: Recursive Traversals and Visual Traversal Shortcuts

0. Purpose of This Phase

Phase 4 teaches how to **visit every node in a binary tree**.

In Phase 3, you learned how to store and build a binary tree using:

- linked nodes
- `Node*` pointers
- `nullptr`
- queues
- level-order construction

Now that the tree exists in memory, the next question is:

How do we move through the tree and visit every node?

That process is called **tree traversal**.

The main beginner idea is:

Traversal means visiting every node exactly once, following a clear rule.

1. Current Progress Analysis

Before Phase 4, you should already understand these ideas:

Previous Idea	Why It Matters in Phase 4
Tree	Traversal happens on a tree.
Binary tree	Phase 4 focuses on left and right children.
Root	Traversal usually starts from the root.
Left child and right child	Traversal rules use left and right subtree order.
Leaf node	Leaf nodes stop recursion because they have no children.
Internal node	Internal nodes may lead to more recursive calls.
Linked representation	Traversal follows child pointers.

Previous Idea	Why It Matters in Phase 4
<code>Node*</code>	Recursive functions receive node addresses.
<code>nullptr</code>	Base case checks whether a child exists.
Queue construction	You can now create a tree, so you can traverse it.

So your current position is:

You can build a binary tree. Now you are learning how to visit every node in that tree using recursive traversal.

2. What Phase 4 Covers

Phase 4 covers these main topics:

1. Meaning of traversal
2. Preorder traversal
3. Inorder traversal
4. Postorder traversal
5. Level-order overview
6. Small dry runs
7. A-G tree dry runs
8. Bracket method
9. Line method
10. Mark-and-trace method
11. Finger tracing method
12. Recursive traversal code
13. System stack and activation records
14. Number of recursive calls
15. Time and space complexity
16. Complete C++ program
17. Common beginner mistakes
18. Practice questions

3. What Does Traversal Mean?

Traversal means:

Visit every node in the tree exactly once.

If a tree has these nodes:

A, B, C, D, E, F, G

A traversal tells us the order in which we visit them.

In an array, traversal is simple because the data is in a straight line:

10 -> 20 -> 30 -> 40

But a tree branches:

```
  A
 /  \
B     C
```

From **A**, we can go left to **B** or right to **C**.

So we need standard rules.

4. Main Types of Tree Traversal

There are four common traversal types in this phase:

Traversal	Order	Simple Meaning
Preorder	Root, Left, Right	Visit root first.
Inorder	Left, Root, Right	Visit root in the middle.
Postorder	Left, Right, Root	Visit root last.
Level order	Top to bottom, left to right	Visit level by level.

The first three are called **depth-first traversals**.

Why?

Because they go deep into a subtree before moving to another subtree.

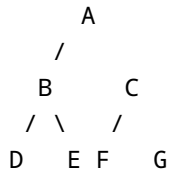
Level order is called **breadth-first traversal**.

Why?

Because it visits across each level before going deeper.

5. Example Tree Used in This Phase

We will use this tree many times:



Important parts:

- A is the root.
- B is the left child of A.
- C is the right child of A.
- D and E are children of B.
- F and G are children of C.
- D, E, F, and G are leaf nodes.
- A, B, and C are internal nodes.

Very important rule:

A traversal rule is applied to every subtree, not only the main root.

For example, if we are doing preorder, then subtree B is also visited using preorder.

Part A: Recursive Depth-First Traversals

6. Preorder Traversal

6.1 Rule

Preorder means:

Root, Left, Right

Simple memory trick:

Preorder = root first.

Or:

Preorder = print before going down.

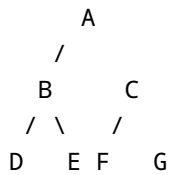
6.2 Meaning

At every node:

1. Visit the current node.
2. Traverse the left subtree.
3. Traverse the right subtree.

6.3 Preorder Dry Run

Use this tree:



Start at **A**.

Step by step:

```
Visit A
Go left to B
Visit B
Go left to D
Visit D
D has no children, go back
Go right to E
Visit E
Go back to A
Go right to C
Visit C
Go left to F
Visit F
Go right to G
Visit G
```

So preorder is:

A B D E C F G

6.4 Preorder on Smaller Trees

Tree 1

```
  A
 /
B  C
```

Preorder:

A B C

Why?

Root first: A

Left: B

Right: C

Tree 2

```
  A
 /
B
```

Preorder:

A B

Tree 3

```
  A
   B
```

Preorder:

A B

Even though B is on the right, preorder still visits root first, then left if it exists, then right.

7. Inorder Traversal

7.1 Rule

Inorder means:

Left, Root, Right

Simple memory trick:

Inorder = root in the middle.

Or:

Inorder = print between left and right.

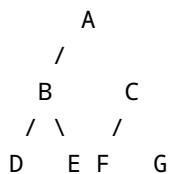
7.2 Meaning

At every node:

1. Traverse the left subtree.
2. Visit the current node.
3. Traverse the right subtree.

7.3 Inorder Dry Run

Use this tree:



Start at A.

Do not print A immediately.

Why?

Because inorder says left first.

Step by step:

```
Go left from A to B
Go left from B to D
D has no left child, so visit D
Go back to B and visit B
Go right from B to E and visit E
Left subtree of A is finished, so visit A
Go right from A to C
Go left from C to F and visit F
Go back to C and visit C
Go right from C to G and visit G
```

So inorder is:

D B E A F C G

7.4 Inorder on Smaller Trees

Tree 1

```
  A
 /
B  C
```

Inorder:

B A C

Why?

Left: B
Root: A
Right: C

Tree 2

```
  A
 /
B
```

Inorder:

B A

Tree 3

```
  A
   \
    B
```

Inorder:

A B

Why?

A has no left child, so visit **A**, then go right to **B**.

7.5 Important Warning About Inorder

Many beginners think:

Inorder always gives sorted output.

This is not always true.

Correct idea:

Inorder gives sorted output only in a Binary Search Tree.

For an ordinary binary tree, inorder is just a visiting order.

8. Postorder Traversal

8.1 Rule

Postorder means:

Left, Right, Root

Simple memory trick:

Postorder = root last.

Or:

Postorder = print after finishing both children.

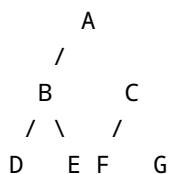
8.2 Meaning

At every node:

1. Traverse the left subtree.
2. Traverse the right subtree.
3. Visit the current node.

8.3 Postorder Dry Run

Use this tree:



Start at **A**.

Do not print **A** immediately.

Why?

Because postorder says root last.

Step by step:

Go left from A to B
Go left from B to D
D has no children, so visit D
Go right from B to E
E has no children, so visit E
Both children of B are done, so visit B
Go right from A to C
Go left from C to F
F has no children, so visit F
Go right from C to G
G has no children, so visit G
Both children of C are done, so visit C
Both children of A are done, so visit A

So postorder is:

D E B F G C A

8.4 Postorder on Smaller Trees

Tree 1

```
  A
 /
B  C
```

Postorder:

B C A

Why?

Left: B
Right: C
Root: A

Tree 2

```
  A
 /
B
```

Postorder:

B A

Tree 3

A
B

Postorder:

B A

9. Comparing Preorder, Inorder, and Postorder

For this tree:

```
      A
     /
    B  C
   / \ /
  D  E F G
```

The traversal outputs are:

Traversal	Rule	Output
Preorder	Root, Left, Right	A B D E C F G
Inorder	Left, Root, Right	D B E A F C G
Postorder	Left, Right, Root	D E B F G C A

Main memory trick:

Preorder = root before children
Inorder = root between children
Postorder = root after children

Part B: Recursive Code

10. Node Structure for Traversal

A binary tree node has:

1. data
2. left child pointer
3. right child pointer

C++ structure:

```
struct Node {  
    char data;  
    Node* left;  
    Node* right;  
  
    Node(char val) {  
        data = val;  
        left = nullptr;  
        right = nullptr;  
    }  
};
```

Explanation:

```
char data;
```

Stores the value, such as `A`, `B`, or `C`.

```
Node* left;
```

Stores the address of the left child.

```
Node* right;
```

Stores the address of the right child.

```
left = nullptr;  
right = nullptr;
```

Means the node starts with no children.

11. The Base Case

Every recursive traversal needs this base case:

```
if (root == nullptr) {  
    return;  
}
```

Simple meaning:

If there is no node here, stop.

Why is this important?

Because leaf nodes have missing children.

Example:

```
  A  
 /  
B
```

Node `B` has no left child and no right child.

So when the function tries to visit `B->left`, that pointer is `nullptr`.

The base case stops the function safely.

Without the base case, the program may try to do this:

```
root->data
```

when `root` is `nullptr`.

That can crash the program.

12. Preorder Recursive Code

Rule:

Root, Left, Right

Code:

```
void preorder(Node* root) {  
    if (root == nullptr) {  
        return;  
    }  
  
    cout << root->data << " ";  
    preorder(root->left);  
    preorder(root->right);  
}
```

The important line is:

```
cout << root->data << " ";
```

It comes before the recursive calls.

So preorder prints the root first.

13. Inorder Recursive Code

Rule:

Left, Root, Right

Code:

```
void inorder(Node* root) {  
    if (root == nullptr) {  
        return;  
    }  
}
```

```
    inorder(root->left);  
    cout << root->data << " ";  
    inorder(root->right);  
}
```

The important line is:

```
cout << root->data << " ";
```

It comes between the left call and the right call.

So inorder prints the root in the middle.

14. Postorder Recursive Code

Rule:

```
Left, Right, Root
```

Code:

```
void postorder(Node* root) {  
    if (root == nullptr) {  
        return;  
    }  
  
    postorder(root->left);  
    postorder(root->right);  
    cout << root->data << " ";  
}
```

The important line is:

```
cout << root->data << " ";
```

It comes after both recursive calls.

So postorder prints the root last.

15. The Only Difference Between the Three Functions

The three traversal functions are almost the same.

The only main difference is the position of the print statement.

Preorder

```
cout << root->data << " ";  
preorder(root->left);  
preorder(root->right);
```

Inorder

```
inorder(root->left);  
cout << root->data << " ";  
inorder(root->right);
```

Postorder

```
postorder(root->left);  
postorder(root->right);  
cout << root->data << " ";
```

Beginner memory trick:

Move the print statement to change the traversal.

Part C: Recursion and the System Stack

16. How Recursion Works

A recursive function is a function that calls itself.

Example:

```
preorder(root->left);
```

This means:

Run preorder again, but now starting from the left child.

When a function calls itself, the current function is paused.

The computer must remember where to return later.

That memory is stored in the **system stack**.

17. System Stack

The system stack is memory used to store unfinished function calls.

For example:

```
preorder(A)
```

Inside `preorder(A)`, we print `A`, then call:

```
preorder(B)
```

But `preorder(A)` is not finished yet.

It still needs to later call:

```
preorder(C)
```

So the computer remembers:

After finishing `B`, return to `A` and continue with `A`'s right child.

That remembered information is kept on the system stack.

18. Activation Record

An activation record is one stored function call on the system stack.

Simple meaning:

One unfinished function call = one activation record.

If the recursion goes deep, more activation records are placed on the stack.

Example path:

A -> B -> D

The stack may contain calls for:

```
preorder(A)
preorder(B)
preorder(D)
```

When **D** finishes, its activation record is removed.

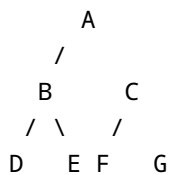
Then control returns to **B**.

When **B** finishes, its activation record is removed.

Then control returns to **A**.

19. Recursion Stack Trace for Preorder

Use this tree:



Preorder rule:

Root, Left, Right

Call flow:

```
preorder(A)
  print A
  preorder(B)
    print B
    preorder(D)
      print D
      preorder(nullptr)
      preorder(nullptr)
    preorder(E)
      print E
      preorder(nullptr)
      preorder(nullptr)
  preorder(C)
    print C
    preorder(F)
      print F
      preorder(nullptr)
      preorder(nullptr)
    preorder(G)
      print G
      preorder(nullptr)
      preorder(nullptr)
```

Output:

A B D E C F G

20. Recursive Calls Including Null Calls

For a binary tree with real nodes, recursive traversal makes:

and:

So total calls are:

$$2N + 1$$

Example:

If $N = 7$, then:

$$\begin{aligned}\text{Total calls} &= 2N + 1 \\ &= 2(7) + 1 \\ &= 15\end{aligned}$$

Why are there null calls?

Because every missing child pointer still gets checked by the recursive function.

Example:

A leaf node has:

```
left = nullptr  
right = nullptr
```

So the traversal calls the function on both missing children, and the base case returns immediately.

Important:

Even though there are $2N + 1$ total calls, the time complexity is still written as:

$$O(N)$$

because Big O ignores constant factors.

Part D: Visual Traversal Shortcuts

21. Why Visual Shortcuts Help

Recursive traversal can feel confusing at first.

Visual shortcuts help you solve traversal questions by hand.

They help you see where each node should be written without tracing every function call.

Phase 4 includes these visual methods:

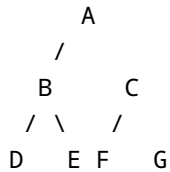
1. Bracket method
2. Line method
3. Mark-and-trace method
4. Finger tracing method

22. Bracket Method

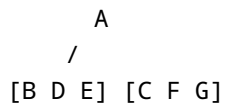
The bracket method means:

Break the tree into root, left subtree, and right subtree.

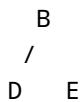
Use this tree:



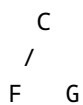
Think of it like this:



The left subtree is:



The right subtree is:



22.1 Bracket Method for Preorder

Preorder rule:

Root, Left, Right

So:

A + preorder(B subtree) + preorder(C subtree)

For B subtree:

B D E

For C subtree:

C F G

Final preorder:

A B D E C F G

22.2 Bracket Method for Inorder

Inorder rule:

Left, Root, Right

So:

inorder(B subtree) + A + inorder(C subtree)

For B subtree:

D B E

For C subtree:

F C G

Final inorder:

D B E A F C G

22.3 Bracket Method for Postorder

Postorder rule:

Left, Right, Root

So:

postorder(B subtree) + postorder(C subtree) + A

For B subtree:

D E B

For C subtree:

F G C

Final postorder:

D E B F G C A

23. The Touch Idea

Imagine walking around the outside boundary of the tree.

Each node can be touched three times:

left side bottom right side

These touches match the three recursive traversal moments.

Touch Point	Traversal
Left side of node	Preorder
Bottom of node	Inorder
Right side of node	Postorder

Simple memory trick:

```
Preorder = left-side touch
Inorder  = bottom touch
Postorder = right-side touch
```

24. Line Method

The line method is a visual shortcut.

You imagine or draw lines from nodes down to a base line.

Then you read the nodes from left to right according to where the traversal mark is placed.

General idea:

Traversal	Where to place/read the mark
Preorder	Left side of node
Inorder	Middle or bottom of node
Postorder	Right side of node

Simple meaning:

The line method turns tree traversal into a left-to-right reading problem.

This is useful for handwritten traversal questions.

25. Mark-and-Trace Method

The mark-and-trace method means:

1. Put marks around each node.
2. Trace around the outside boundary of the tree.
3. Write a node when you touch the correct mark.

Use this rule:

Traversal	Mark to use
Preorder	Left mark
Inorder	Bottom mark
Postorder	Right mark

25.1 Preorder with Mark-and-Trace

Write a node when you touch its left mark.

This means the node is written early.

Output for A-G tree:

A B D E C F G

25.2 Inorder with Mark-and-Trace

Write a node when you touch its bottom mark.

This means the node is written after the left subtree but before the right subtree.

Output for A-G tree:

D B E A F C G

25.3 Postorder with Mark-and-Trace

Write a node when you touch its right mark.

This means the node is written late, after both children.

Output for A-G tree:

D E B F G C A

26. Finger Tracing Method

Finger tracing is similar to mark-and-trace, but you do not draw marks.

You use your finger or eyes to trace around the tree.

When your finger reaches the correct side of the node, you say or write the node.

Traversal	When to say the node
Preorder	When your finger reaches the left side of the node
Inorder	When your finger passes under the node
Postorder	When your finger reaches the right side of the node

Simple meaning:

Finger tracing is mark-and-trace without drawing marks.

This is useful during exams when you want a fast method.

27. Boundary Trace Function

This function shows all three traversal moments:

```
void traceBoundary(Node* root) {
    if (root == nullptr) {
        return;
    }

    cout << "Left Side (Pre): " << root->data << endl;

    traceBoundary(root->left);

    cout << "Bottom Side (In): " << root->data << endl;

    traceBoundary(root->right);
}
```

```
    cout << "Right Side (Post): " << root->data << endl;  
}
```

The function does not perform only one traversal.

Instead, it shows when each traversal would print the node.

27.1 Meaning of Each Print

```
cout << "Left Side (Pre): " << root->data << endl;
```

This is the preorder moment.

```
cout << "Bottom Side (In): " << root->data << endl;
```

This is the inorder moment.

```
cout << "Right Side (Post): " << root->data << endl;
```

This is the postorder moment.

27.2 Beginner Memory Trick

```
First time reaching node  -> Preorder  
After left subtree       -> Inorder  
After right subtree      -> Postorder
```

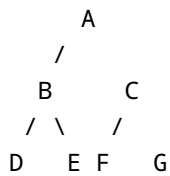
Part E: Level Order Overview

28. What Is Level Order Traversal?

Level order means:

Visit nodes level by level, from left to right.

Use this tree:



Level 1:

A

Level 2:

B C

Level 3:

D E F G

So level order is:

A B C D E F G

28.1 Level Order Memory Trick

Level order reads the tree like a book: top to bottom, left to right.

28.2 Important Note

Recursive preorder, inorder, and postorder are depth-first traversals.

Level order is different because it is breadth-first.

Usually, level order uses a queue.

You will study this more deeply in Phase 5.

Part F: Complete C++ Program

29. Complete Program for Recursive Traversals

```
#include <iostream>
using namespace std;

struct Node {
    char data;
    Node* left;
    Node* right;

    Node(char val) {
        data = val;
        left = nullptr;
        right = nullptr;
    }
};

void preorder(Node* root) {
    if (root == nullptr) {
        return;
    }

    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}

void inorder(Node* root) {
    if (root == nullptr) {
        return;
    }

    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}

void postorder(Node* root) {
    if (root == nullptr) {
        return;
    }

    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
```

```

void traceBoundary(Node* root) {
    if (root == nullptr) {
        return;
    }

    cout << "Left Side (Pre): " << root->data << endl;
    traceBoundary(root->left);
    cout << "Bottom Side (In): " << root->data << endl;
    traceBoundary(root->right);
    cout << "Right Side (Post): " << root->data << endl;
}

int main() {
    Node* root = new Node('A');

    root->left = new Node('B');
    root->right = new Node('C');

    root->left->left = new Node('D');
    root->left->right = new Node('E');

    root->right->left = new Node('F');
    root->right->right = new Node('G');

    cout << "Preorder: ";
    preorder(root);
    cout << endl;

    cout << "Inorder: ";
    inorder(root);
    cout << endl;

    cout << "Postorder: ";
    postorder(root);
    cout << endl;

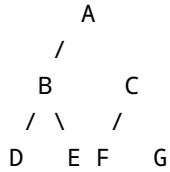
    cout << "\nBoundary trace:" << endl;
    traceBoundary(root);

    return 0;
}

```

30. Tree Created by the Program

The program creates this tree:



This part creates the root:

```
Node* root = new Node('A');
```

This creates the left and right children of A:

```
root->left = new Node('B');  
root->right = new Node('C');
```

This creates the children of B:

```
root->left->left = new Node('D');  
root->left->right = new Node('E');
```

This creates the children of C:

```
root->right->left = new Node('F');  
root->right->right = new Node('G');
```

31. Expected Output

```
Preorder: A B D E C F G  
Inorder: D B E A F C G  
Postorder: D E B F G C A
```

The boundary trace output will show each node at its preorder, inorder, and postorder moments.

Part G: Complexity

32. Time Complexity

For preorder, inorder, and postorder:

Time complexity = $O(N)$

Why?

Because every real node is visited once.

If there are 7 nodes, we visit 7 nodes.

If there are 100 nodes, we visit 100 nodes.

So traversal grows directly with the number of nodes.

33. Space Complexity

For recursive traversal:

Space complexity = $O(H)$

Here:

H = height of the tree

Why?

Because recursion uses the system stack.

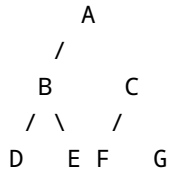
The maximum number of unfinished function calls depends on the deepest path in the tree.

That deepest path is the height.

34. Best and Worst Stack Space Cases

Balanced Tree

Example:

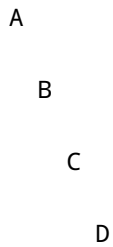


The height is small.

So the recursion stack is smaller.

Skewed Tree

Example:



The height is large.

The tree behaves like a linked list.

So the recursion stack can grow up to $O(N)$ in the worst case.

35. Complexity Summary

Traversal	Time	Extra Space
Preorder	$O(N)$	$O(H)$
Inorder	$O(N)$	$O(H)$
Postorder	$O(N)$	$O(H)$

Traversal	Time	Extra Space
Level order	$O(N)$	Usually $O(N)$ with a queue

Part H: Common Beginner Mistakes

36. Mistake 1: Memorizing Outputs Instead of Rules

Wrong approach:

Trying to memorize `A B D E C F G` without understanding why.

Correct approach:

Apply the traversal rule to every subtree.

Example:

Preorder means:

Root, Left, Right

So for subtree `B`:

`B D E`

Do not memorize randomly.

Use the rule.

37. Mistake 2: Printing Null Nodes

Wrong idea:

Print even when the pointer is null.

Correct idea:

```
if (root == nullptr) {  
    return;  
}
```

A null pointer means no node exists there.

So nothing should be printed.

38. Mistake 3: Forgetting the Base Case

Without the base case:

```
if (root == nullptr) {  
    return;  
}
```

The function may try to access a missing node.

That can crash the program.

Always write the base case first.

39. Mistake 4: Thinking Inorder Always Sorts Values

Wrong idea:

Inorder traversal always produces sorted values.

Correct idea:

Inorder produces sorted values only in a Binary Search Tree.

For a normal binary tree, inorder is simply:

```
Left, Root, Right
```

40. Mistake 5: Ignoring the Call Stack

Beginners sometimes think recursion jumps randomly.

Correct idea:

Recursion pauses a function, goes deeper, then returns to where it paused.

Example:

After finishing **D**, control returns to **B**.

After finishing **E**, control returns to **B** again.

After **B** is done, control returns to **A**.

The system stack remembers these return points.

41. Mistake 6: Applying the Rule Only to the Main Root

Wrong idea:

Use Root, Left, Right only at **A**.

Correct idea:

Use the traversal rule at every node and every subtree.

For preorder:

```
A subtree: A B D E C F G
B subtree: B D E
C subtree: C F G
```

Every subtree follows the same rule.

Part I: Key Vocabulary Table

42. Phase 4 Vocabulary

Term	Simple Meaning
Traversal	Visiting every node exactly once
Preorder	Root, Left, Right
Inorder	Left, Root, Right
Postorder	Left, Right, Root
Level order	Visit level by level, left to right
Recursive function	A function that calls itself
Base case	The stopping condition
<code>nullptr</code>	Means no node exists there
System stack	Memory that stores unfinished function calls
Activation record	One stored function call on the stack
Depth-first traversal	Go deep into subtrees before moving across
Breadth-first traversal	Visit across each level first
Bracket method	Split tree into root, left subtree, and right subtree
Line method	Use visual lines to read traversal order
Mark-and-trace method	Mark nodes and trace around the tree
Finger tracing	Trace the tree with your finger or eyes
<code>O(N)</code>	Time grows with number of nodes
<code>O(H)</code>	Stack space grows with tree height

Part J: Phase 4 Summary

43. Main Summary

By the end of Phase 4, you should know:

- Traversal means visiting every node exactly once.
- Preorder means Root, Left, Right.
- Inorder means Left, Root, Right.

- Postorder means Left, Right, Root.
 - Level order means top to bottom, left to right.
 - Recursive traversal needs a base case.
 - The base case is usually `root == nullptr`.
 - The only major difference between preorder, inorder, and postorder code is where the print statement is placed.
 - Recursion uses the system stack.
 - Each unfinished recursive call is an activation record.
 - For `N` nodes, recursive traversal makes `2N + 1` calls including null calls.
 - Traversal time is `O(N)`.
 - Recursive stack space is `O(H)`.
 - Visual methods help solve traversal problems by hand.
-

44. Final Memory Tricks

Traversal Rule Trick

```
Preorder  = Root first
Inorder   = Root middle
Postorder = Root last
```

Print Statement Trick

```
Preorder = print before recursive calls
Inorder  = print between recursive calls
Postorder = print after recursive calls
```

Touch Method Trick

```
Preorder = left side of node
Inorder  = bottom of node
Postorder = right side of node
```

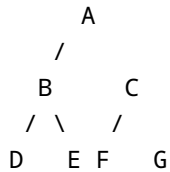
Recursion Trick

```
A recursive traversal means:
Do the same traversal on the left subtree.
Do the same traversal on the right subtree.
Stop when the pointer is nullptr.
```

Part K: Practice Questions

45. Practice Tree

Use this tree:



Answer these questions:

1. What is the preorder traversal?
2. What is the inorder traversal?
3. What is the postorder traversal?
4. What is the level-order traversal?
5. Which traversal visits the root first?
6. Which traversal visits the root in the middle?
7. Which traversal visits the root last?
8. What is the base case in recursive traversal?
9. Why do we check `root == nullptr`?
10. What is the time complexity of recursive traversal?
11. What is the stack space complexity?
12. What does the system stack store?
13. What is an activation record?
14. Where is the print statement in preorder?
15. Where is the print statement in inorder?
16. Where is the print statement in postorder?
17. When does inorder produce sorted output?
18. What does the bracket method do?
19. What does the mark-and-trace method do?
20. What is the finger tracing method?

46. Practice Answers

Answer 1

Preorder means:

Root, Left, Right

So:

A B D E C F G

Answer 2

Inorder means:

Left, Root, Right

So:

D B E A F C G

Answer 3

Postorder means:

Left, Right, Root

So:

D E B F G C A

Answer 4

Level order means level by level:

A B C D E F G

Answer 5

Preorder visits the root first.

Answer 6

Inorder visits the root in the middle.

Answer 7

Postorder visits the root last.

Answer 8

The base case is:

```
if (root == nullptr) {  
    return;  
}
```

Answer 9

We check `root == nullptr` because a missing child means there is no node to visit.

The function should stop safely.

Answer 10

The time complexity is:

```
O(N)
```

because every real node is visited once.

Answer 11

The recursive stack space is:

$O(H)$

where H is the height of the tree.

Answer 12

The system stack stores unfinished recursive function calls.

Answer 13

An activation record is one stored function call on the system stack.

Answer 14

In preorder, the print statement comes before the recursive calls.

```
cout << root->data << " ";  
preorder(root->left);  
preorder(root->right);
```

Answer 15

In inorder, the print statement comes between the left and right recursive calls.

```
inorder(root->left);  
cout << root->data << " ";  
inorder(root->right);
```

Answer 16

In postorder, the print statement comes after both recursive calls.

```
postorder(root->left);  
postorder(root->right);  
cout << root->data << " ";
```

Answer 17

Inorder produces sorted output only in a Binary Search Tree.

Answer 18

The bracket method splits the tree into:

```
root
left subtree
right subtree
```

Then it applies the traversal rule to each part.

Answer 19

The mark-and-trace method puts marks around nodes and traces around the tree.

You write a node when you touch the correct mark.

Answer 20

Finger tracing is like mark-and-trace, but without drawing marks.

You trace the tree with your finger or eyes and write nodes at the correct touch point.

47. Final Beginner Checklist Before Phase 5

Before moving to Phase 5, make sure you can answer these:

- What does traversal mean?
- What is preorder traversal?
- What is inorder traversal?
- What is postorder traversal?
- What is level order traversal?
- What is the preorder output of the A-G tree?
- What is the inorder output of the A-G tree?
- What is the postorder output of the A-G tree?
- Why does recursive traversal need a base case?

- What does `nullptr` mean in traversal?
- Where does the print statement go in preorder?
- Where does the print statement go in inorder?
- Where does the print statement go in postorder?
- What is the system stack?
- What is an activation record?
- Why is traversal time $O(N)$?
- Why is recursive stack space $O(H)$?
- What are the line method, mark-and-trace method, and finger tracing method?

If you can answer these questions, you understand Phase 4 well.

48. Final Phase 4 Memory Statement

Phase 4 is mainly about this:

Once a tree is built, traversal tells us how to visit every node.

There are three main recursive depth-first traversal rules:

```
Preorder  = Root, Left, Right
Inorder   = Left, Root, Right
Postorder = Left, Right, Root
```

The code is almost the same for all three.

Only the position of the print statement changes.

Recursion uses the system stack, and visual shortcuts help you solve traversal questions by hand.

Once this is clear, Phase 5 becomes easier because Phase 5 replaces the hidden recursion stack with your own explicit stack and queue.